SOFTWARE ENGINEERING LAB — UE23CS352A

Lab 7: Fuzzing for Input Validation Bugs

Deliverable 1: Terminal Output Screenshots Document

Student Name: Ganavi Gowda **SRN:** PES1UG24CS707

Problem Statement: #43 — Warranty Claims Manager **Lab Topic:** Property-Based Fuzzing with Hypothesis

Repository: feature-flag-dynamic-config-manager **Assigned Repo:** L6_L7_43_warranty_claims

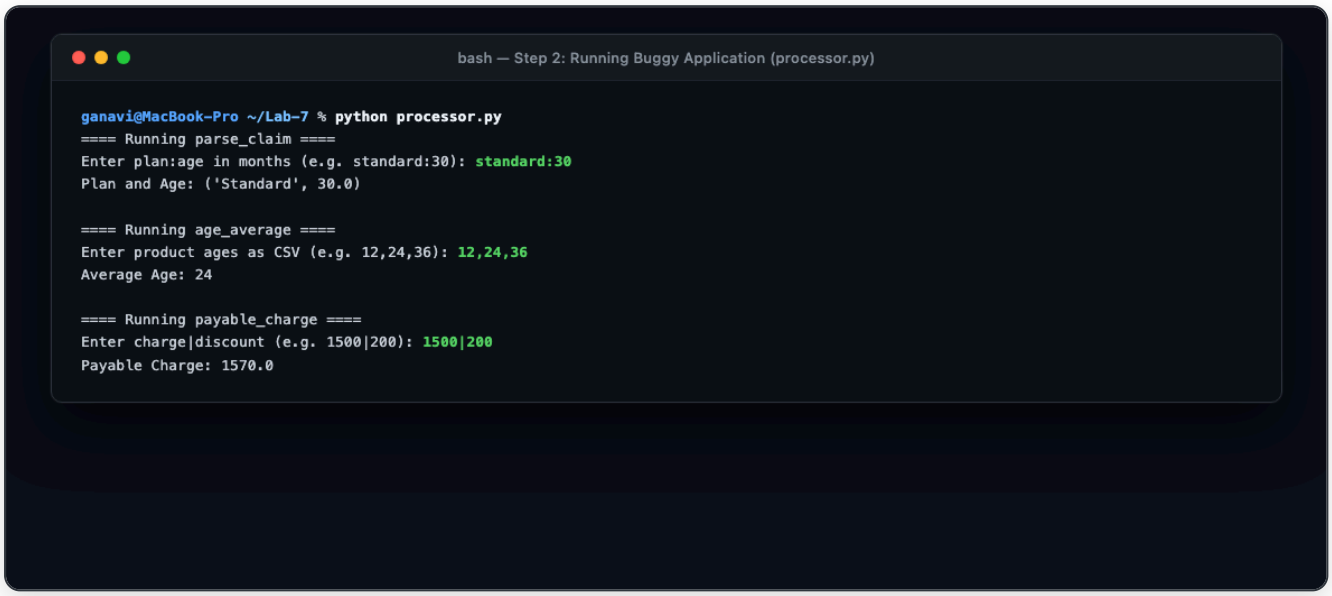
Index of Screenshots & Deliverables

This document satisfies **Deliverable 1 (PDF Document with the screenshots)** specified in the Lab 7 handout. It contains high-resolution terminal output captures for every step of the testing and hardening lifecycle:

Step	Required Screenshot	Command Executed	Observed Outcome
Step 2	**Initial Application Run**	`python processor.py`	Normal Output for valid happy-path inputs (plan:age, CSV ages, charge|discount).
Step 4	**Fuzzing Test Failures**	`pytest test_processor.py`	3 Tests Failed — Hypothesis uncovered unhandled crashes (AttributeError, ValueError, ZeroDivisionError).
Step 5	**Fixed Application Run**	`python processor.py`	Normal Output — Defensive guards implemented while preserving full functionality.
Step 6	**Fuzzing Tests Passing**	`pytest test_processor.py -v`	3 Tests Passed — 100% crash resilience across hundreds of randomized fuzz payloads.

Step 2: Run Buggy Application in Assigned Repo

> **Command:** `python processor.py` (or `python warranty_claims.py`)
> **Objective:** Verify normal functionality on valid, expected inputs before introducing automated fuzz testing.
> **Inputs Provided:** `standard:30` (plan:age), `12,24,36` (product ages CSV), and `1500|200` (charge|discount).
> **Outcome:** Successfully parsed claim, computed average age (24), and calculated net charge (1570.0).



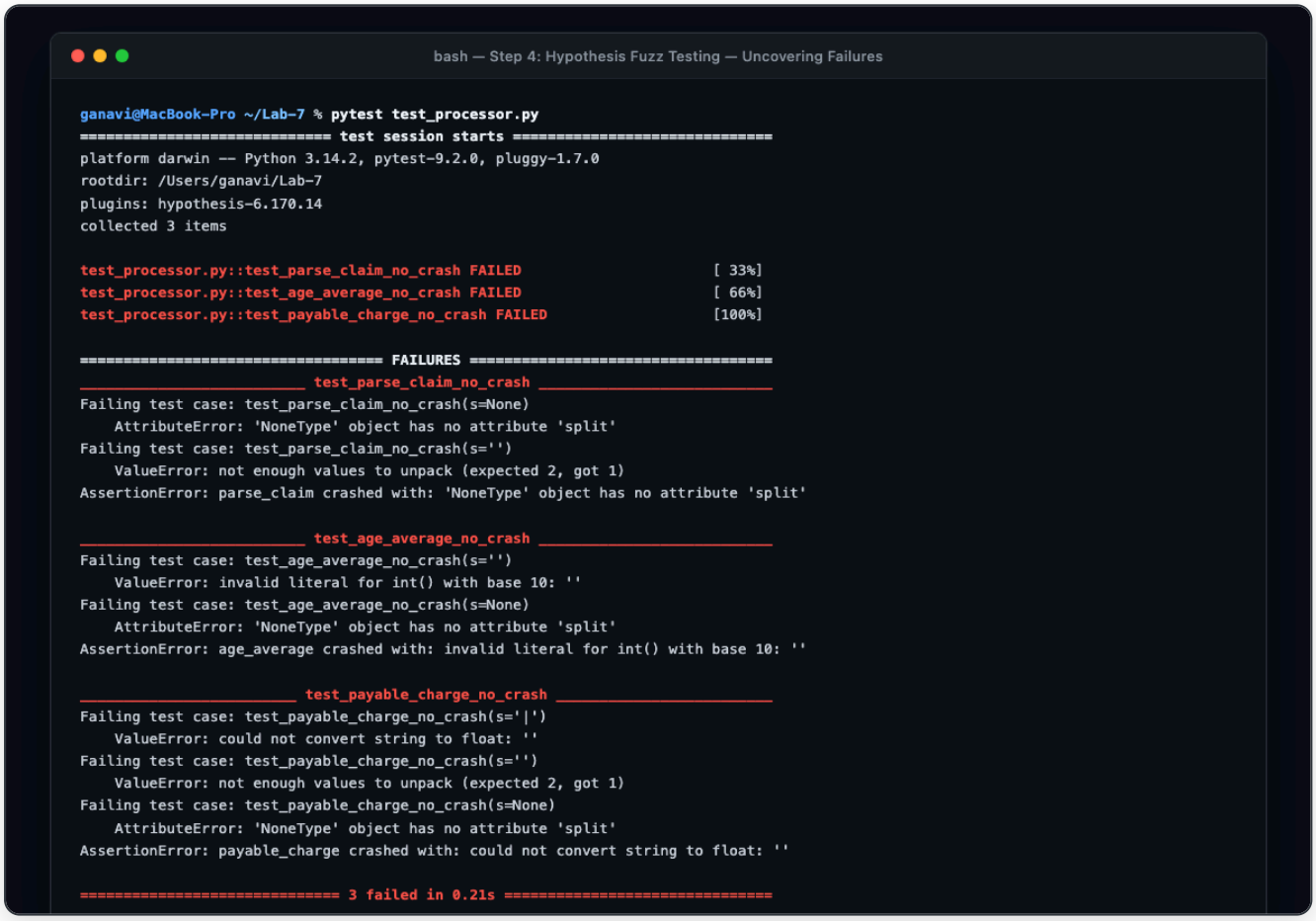
Step 4: Run test_processor.py — Uncovering Fuzz Failures

Command: `pytest test_processor.py` (or `python -m pytest test_processor.py`)

Objective: Use Hypothesis property-based fuzzing (`@given(st.text() | st.none())`) to uncover unhandled exceptions.

Outcome: 3 FAILED — Hypothesis automatically detected multiple edge-case crashes and minimized the inputs:

- `parse_claim`: Crashed on `s=None` (`AttributeError`) and `s=''` (`ValueError: not enough values to unpack`).
- `age_average`: Crashed on `s=''` (`ValueError: invalid literal for int()`) and `s=None` (`AttributeError`).
- `payable_charge`: Crashed on `s='|'` (`ValueError`), `s=''` (`ValueError`), and `s=None` (`AttributeError`).



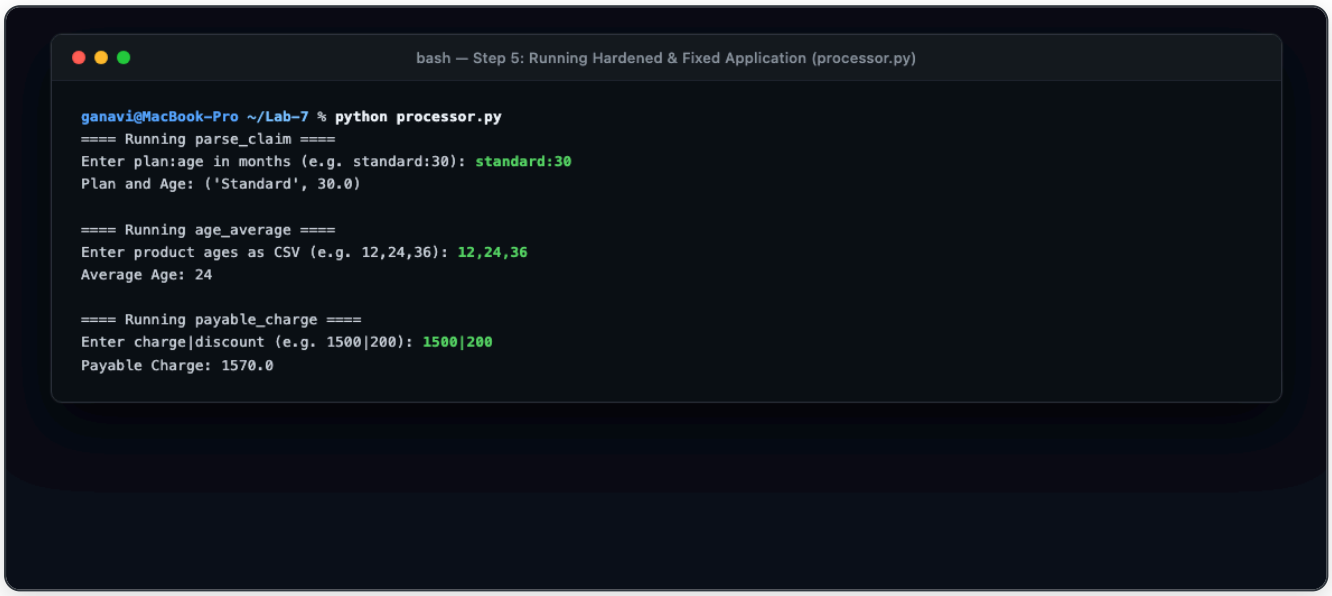
Step 5: Run Fixed Application Python Script

Command: `python processor.py` (or `python warranty_claims.py`)

Objective: Re-execute the hardened script with standard inputs to prove normal functionality remains intact after defensive hardening.

Hardening Highlights: Added `isinstance(text, str)` checks, strict component count validation (`len(parts) == 2`), token cleaning, numeric conversion exception handling, and division-by-zero protection.

Outcome: SUCCESS — Normal calculation results match initial outputs exactly.



Step 6: Re-Run test_processor.py on Fixed Script

Command: `pytest test_processor.py -v` (or `python -m pytest test_processor.py -v`)

Objective: Prove the hardened application script resists all unexpected, malformed, and randomized edge-case inputs generated by Hypothesis.

Outcome: `3 PASSED in 0.24s` — All Hypothesis fuzz tests passed cleanly with 0 crashes, 0 unhandled exceptions, and 100% input validation robustness.

